

Matrix Multiplication on Graphics Processing Units

PhD Course Project Report: GPU-Accelerated Computing for AI and Computer Vision

Fabio A. Ciraci

GitHub Repository: GPU-Matrix-Multiplication-Project

July 14, 2026

Abstract

Matrix multiplication ($C = A \times B$) serves as a fundamental building block in modern scientific computing, deep learning, and computer vision. Achieving peak theoretical performance on modern Graphics Processing Units (GPUs) requires overcoming severe memory bandwidth and instruction-level bottlenecks. This report details the implementation, optimization, and evaluation of various parallel matrix multiplication. The first tests were executed are naive CPU implementations of Matrix Multiplication algorithms, while the last is the implementation of a CUDA specific multiplication via the CuPy lib. All the code is in Python. LLMs were used to support code and the report writing.

1 Introduction

Matrix multiplication is a foundational operation in modern computing, serving as the computational backbone for diverse applications ranging from climate modeling and scientific simulations to the training and inference of Large Language Models (LLMs). Among these, the General Matrix Multiply (GEMM) operation, which computes the product of two dense matrices, is especially critical. Because of its high computational intensity, GEMM is the primary performance bottleneck in the linear algebra routines that underpin modern machine learning workloads.

To illustrate, the multiplication of two 2×2 matrices is defined as:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x & y \\ z & w \end{bmatrix} = \begin{bmatrix} ax + bz & ay + bw \\ cx + dz & cy + dw \end{bmatrix}$$

While conceptually straightforward, a naive implementation of this algorithm scales with a time complexity of $\mathcal{O}(N^3)$ for $N \times N$ square matrices. This cubic complexity makes hardware-level optimization imperative for large-scale applications, particularly when attempting to fully saturate the massive parallel processing capabilities of modern Graphics Processing Units (GPUs).

In this report, we investigate and compare several implementations of matrix multiplication. We begin with the naive CPU baseline and progressively introduce advanced optimization techniques, including shared memory tiling and memory-access optimizations such as matrix transposition.

2 Implementation & Optimization Strategies

This section forms the core technical contribution of your report. Describe each implementation variant step-by-step.

Given the interpreted nature of Python, we leverage the `Numba` library to JIT-compile our matrix multiplication functions for CPU execution.

To JIT-compile a function, we use the `@jit` decorator from Numba. This allows us to write Python code that is compiled to machine code at runtime, significantly improving performance.

2.1 Baseline CPU Implementation

The most straightforward approach to matrix multiplication is the naive algorithm, which utilizes three nested loops to compute the dot product of rows from matrix A and columns from matrix B . While conceptually simple, this baseline implementation is highly inefficient on modern CPU architectures. Because matrix B is accessed column-by-column, the memory access pattern violates spatial locality, resulting in frequent cache misses that severely bottleneck performance.

```
1 import numpy as np
2
3 @jit(nopython=True)
4 def naive_approach(A, B, C):
5     for i in range(N):
6         for j in range(N):
7             for k in range(N):
8                 C[i][j] += A[i][k] * B[k][j]
```

Listing 1: Baseline Naive CPU Matrix Multiplication in Python

2.2 Exploiting the Locality principle

Using the locality principle, we can improve cache performance by accessing data in a more cache-friendly manner. CPUs pre-load the data into cache line by line, in the naive implementation the A matrix is accessed row-wise, while the B matrix is accessed column-wise, which can lead to cache misses. It is then possible to invert the order of the loops to access the B matrix row-wise, which is more cache-friendly and can lead to better performance.

```
1 @jit(nopython=True)
2 def exploiting_locality(A, B, C):
3     for i in range(N):
4         for k in range(N):
5             for j in range(N):
6                 C[i][j] += A[i][k] * B[k][j]
```

Listing 2: Baseline Naive CPU Matrix Multiplication with Locality Principle in Python

2.3 Transposed Approach

In the transposed approach, we first transpose the second matrix B before performing the multiplication. This allows us to access both matrices in a row-wise manner, which is more cache-friendly and can lead to better performance.

```
1 @jit(nopython=True)
2 def transposed_approach(A, B, C):
3     for i in range(N):
4         for j in range(N):
5             for k in range(N):
6                 C[i][j] += A[i][k] * B[j][k]
7
8 ## In the main code
9 B_t = np.transpose(B).copy()
10 transposed_approach(A, B_t, C)
```

Listing 3: Baseline Naive CPU Matrix Multiplication with Locality Principle in Python

2.4 Tiled Matrix Multiplication (Shared Memory)

In Tiled Matrix Multiplication, we divide the matrices into smaller sub-matrices (tiles) that fit into the shared memory of the GPU. This approach reduces the number of global memory

accesses, which are slower than shared memory accesses, and allows for better utilization of the GPU’s parallel processing capabilities.

```

1 @jit(nopython=True)
2 def tiling_approach(A, B, C, block_size):
3     for ih in range(0, N, block_size):
4         for kh in range(0, N, block_size):
5             for jh in range(0, N, block_size):
6                 # min is necessary in case the matrix is smaller than the sum of the
7                 # the block and the already reached position
8                 for i in range(ih, min(ih + block_size, N)):
9                     for k in range(kh, min(kh + block_size, N)):
10                        for j in range(jh, min(jh + block_size, N))
11                        C[i][j] += A[i][k] * B[k][j]

```

Listing 4: Tiled Matrix Multiplication

2.5 GPU Optimization with CuPy

Specialized GPU libraries like CuPy can be used to perform matrix multiplication on the GPU with minimal code changes.

```

1 import cupy as cp
2
3 A = np.random.rand(N, N).astype(np.float32)
4 B = np.random.rand(N, N).astype(np.float32)
5 C = np.zeros((N, N), dtype=np.float32)
6
7 C = cp.dot(A, B)
8 cp.cuda.Stream.null.synchronize()

```

Listing 5: CuPy Implementation

3 Hardware Architecture

The hardware architecture used for testing and evaluation is both a modern CPU and NVIDIA GPU. The local CPU is an Intel Core i7-12700H with 14 cores (6 performance cores and 8 efficiency cores) and a base clock of 2.3 GHz. Local GPU for testing is an NVIDIA RTX 4050 Laptop GPU with 6GB of GDDR6 memory, 2560 CUDA cores, and a base clock of 1.5 GHz.

4 Experiments and Results

As shown in *Table 1*, the naive CPU approach without JIT compilation yielded the worst performance. Conversely, the CPU-based tiling approach using a block size of 64 achieved the best performance overall, even comparable to the GPU-accelerated CuPy implementation.

To visualize these performance trends, *Figure 1* and *Figure 2* illustrate the execution times for all tested approaches under JIT and non-JIT compilation, respectively. These results clearly demonstrate the substantial performance benefits of both JIT compilation and GPU acceleration for matrix multiplication tasks.

Ultimately, the tiling approach with a block size of 64 stood out as the most effective configuration. By optimizing cache locality, this specific setup maximized CPU efficiency to the point of rivaling the dedicated GPU CuPy implementation, making a block size of 64 the optimal choice for these experiments.

Table 1: Results of naive approaches with and without JIT-compiler ($N = 512$)

Configuration/Approach	Without JIT (ms)	With JIT (ms)
Naive	41834.25	794.34
Locality Exploitation	40259.05	87.05
Transpose Naive	39726.54	117.26
CuPy (GPU)	69.95	69.95
Tiling (Block Dimension)		
Tile 8	42781.91	214.82
Tile 16	40548.84	73.65
Tile 32	39608.49	63.54
Tile 64	38973.51	62.58
Tile 128	39512.94	70.06
Tile 256	39515.65	73.93
Tile 512	39332.61	64.53
Tile 1024	40025.15	63.41

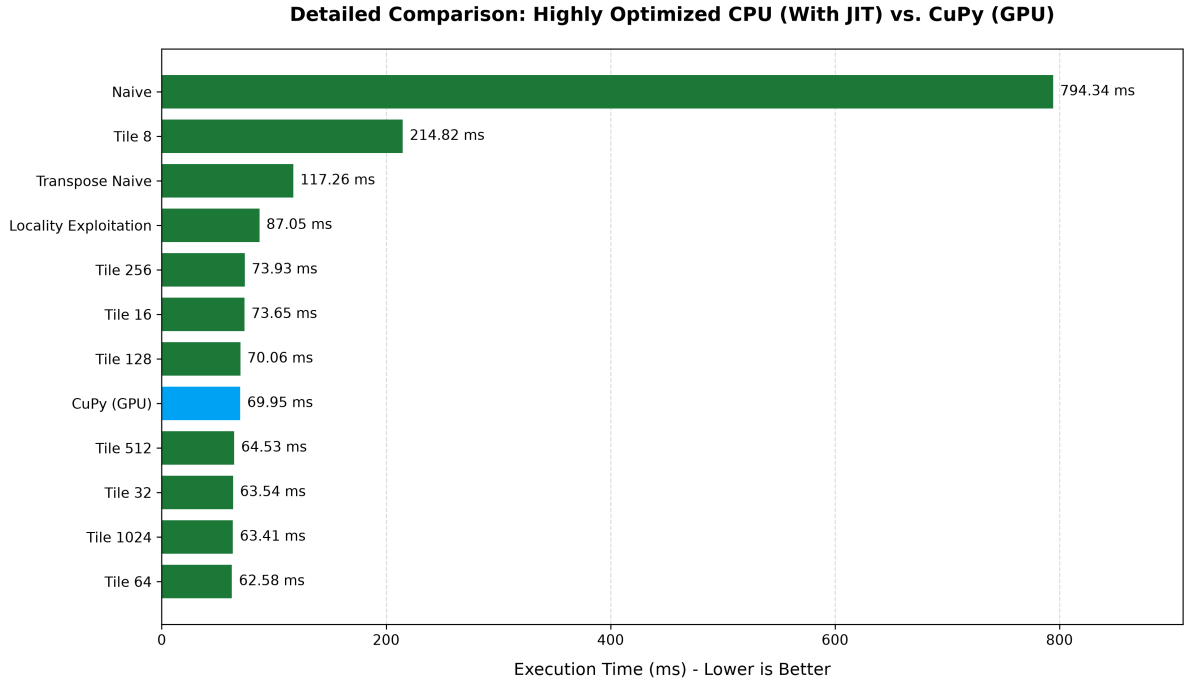


Figure 1: Comparison of temporal performance (ms) with JIT compiler, ordered by execution time.

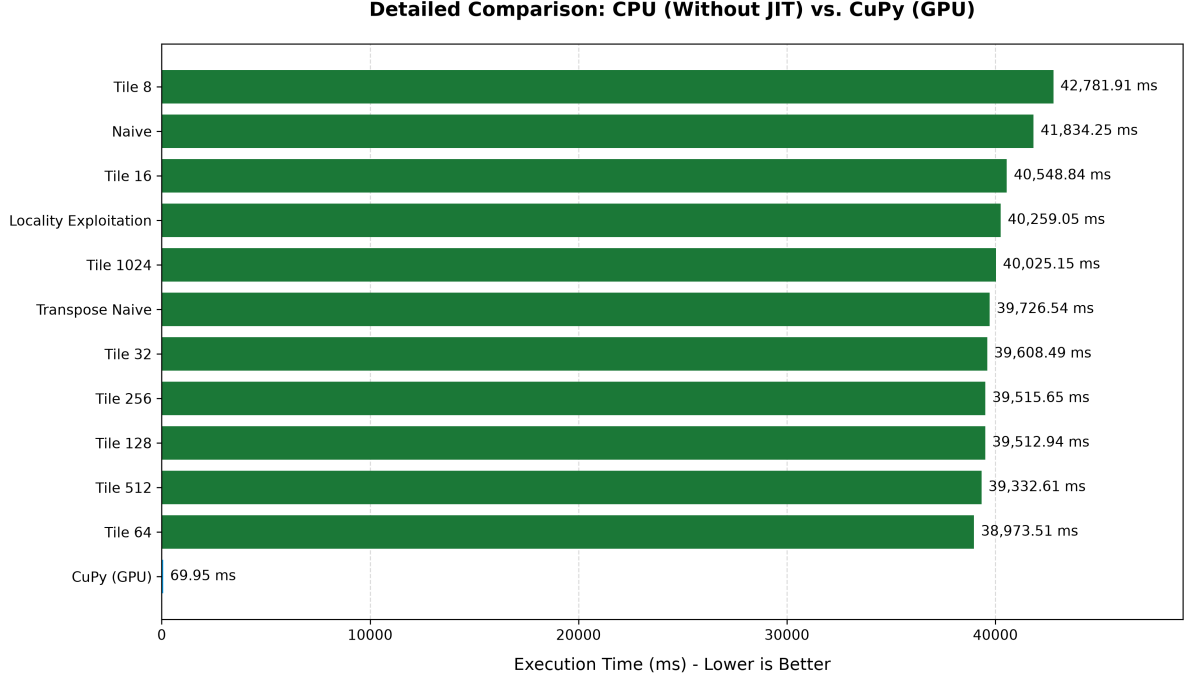


Figure 2: Comparison of temporal performance (ms) with and without JIT compiler, ordered by execution time.

5 Conclusion and Future Work

This report has explored various implementations of matrix multiplication, from naive CPU approaches to optimized GPU techniques using CuPy. The findings highlight the significant performance gains achievable through JIT compilation and GPU acceleration, while also emphasizing the importance of considering data transfer overheads in GPU computations. Future work that could be interesting to explore may take in consideration advanced GPU optimization techniques in particular on multi-GPU systems, and to investigate the impact of different matrix sizes and shapes on performance.